

Flancos señalados

Captura: flanco que termina el ciclo en IDLE con `start=1` y `capture_en=1`. Se guardan `rs1_q=1`, `rs2_q=2`, `rd_q=5` y `op_q=ADD`, y la FSM pasa a `FETCH_OPS`.

Lectura: flanco que termina `FETCH_OPS`. Con `regA_idx=1` y `regB_idx=2`, el banco actualiza sus salidas: `rf_rs1_data=10` y `rf_rs2_data=20`.

Escritura: flanco que termina `WRITEBACK`. Con `rf_we=1` y `regA_idx=5`, el banco guarda `result` en R5 y la FSM vuelve a `IDLE`.

Valores por estado

Estado	regA_idx	rf_rs1_data	result
FETCH_OPS	1 (<code>rs1_q</code> , <code>rf_we=0</code>)	0 (todavía no se leyó)	0(no válido)
EXECUTE	1	10	30
WRITEBACK	5 (<code>rd_q</code> , <code>rf_we=1</code>)	10	30

Como su salida depende sólo de sus entradas actuales y no necesita esperar un flanco de reloj, debido a que la ALU es combinacional, apenas el banco entrega los operandos (`rf_rs1_data=10` y `rf_rs2_data=20`), `result` pasa a valer 30, que permanece estable durante `WRITEBACK`, porque los operandos leídos no cambian.

Por otro lado, el banco de registros es secuencial ,es decir, solo guarda un dato en un flanco ascendente con `rf_we=1`. Se da recién durante `WRITEBACK`, y la escritura se concreta en el flanco que termina ese estado. De esta forma, `result` ya está calculado y estable cuando llega el flanco de escritura, y ese es el valor que se almacena en R5.

Finalmente , en `WRITEBACK` `regA_idx` pasa de `rs1_q` a `rd_q` (5), pero `rf_rs1_data` no cambia rápido, porque la lectura es síncrona. Y el puerto A solo se actualiza en el siguiente flanco. Esto evita que `result` se modifique antes de ser escrito. Después de la escritura, el puerto A lee el contenido anterior de R5, por lo que `result` puede cambiar y eso es esperable.